

BUILDING SECURE & SCALABLE DATATABLES APIS IN COLDFUSION

Session goal: Show how a Bootstrap 5 + jQuery DataTables front end can talk to a ColdFusion CFC endpoint that validates input, protects the SQL layer, returns the exact DataTables JSON contract, and still performs well under real paging, sorting, and search load.

SPEAKER BIOGRAPHY

Larry C. Lyons - larryclyons@gmail.com

- I have spent more than 20 years in ColdFusion development, building secure, scalable, and maintainable ColdFusion applications that perform reliably in enterprise environments
- Much of my work has focused on designing, maintaining, and modernizing ColdFusion applications using server-side architecture and API-driven integrations.

OPENING

We are walking through how to build secure, scalable DataTables APIs in ColdFusion. The point of this session is not just to make DataTables display rows. You can do that with a static JSON file. The point is to build the kind of endpoint you can put behind a real application: one that validates the browser request, protects the database, handles paging and searching, and returns a predictable response every time.

The demo files are deliberately simple: index.cfm provides the browser experience, and Employees.cfc provides the server-side JSON endpoint. That simplicity is useful because it lets us focus on the exact seam where many production bugs and security issues happen: the boundary between browser-supplied parameters and database-backed SQL.

THE PROBLEM

Client-side DataTables library works beautifully when your dataset is small. You load the rows, DataTables sorts them, filters them, paginates them, and everybody is happy. But that breaks down quickly when the row count grows, when the browser has to hold thousands or hundreds of thousands of records, or when multiple users are running searches at the same time.

The failure mode is predictable: slow initial load, heavy memory use in the browser, inefficient sorting, and ultimately an application that feels broken even when the database itself is perfectly capable of returning a page of results quickly.

The solution is server-side processing. Instead of giving the browser everything, we let the browser ask for exactly one page of data. ColdFusion receives the front-end request, validates it, runs a bounded query, and returns only the rows needed for the current draw.

WHAT IS DATATABLES?

DataTables (<http://datatable.net/>) is a JavaScript HTML table enhancing library.

It is a highly flexible tool built on the foundations of progressive enhancement. In plain terms, it lets you start with a normal HTML table and progressively add the interactive behavior users expect in a modern enterprise application.

- It provides built-in searching, sorting, filtering, pagination, and responsive layouts.
- It supports both client-side and server-side processing models.
- It is widely used in enterprise applications for displaying large datasets.
- It integrates cleanly with JSON APIs and many front-end interfaces like Bootstrap etc.

For this paper, the key point is the difference between client-side and server-side processing. Client-side processing is convenient when the dataset is small. Server-side

processing is the pattern we use when the dataset is large, the query logic belongs on the server, and the browser should receive only the current page of results.

DEMO ARCHITECTURE

The architecture is straightforward: Browser → index.cfm → DataTables AJAX request → com/Employees.cfc?method=getEmployeesJSON → database → JSON response.

In index.cfm, the table itself is ordinary Bootstrap-friendly HTML. The important detail is that the tbody is empty. That is intentional. The rows are not rendered by ColdFusion on the initial page load; they are retrieved by DataTables after initialization.

```
<table id="employees" class="table table-sm table-striped table-hover
    table-bordered compact dt-center rounded">
  <thead>
    <tr>
      <th>ID</th>
      <th>Name</th>
      <th>Position</th>
      <th>Salary</th>
      <th>Office</th>
      <th>Extension</th>
      <th>Start date</th>
    </tr>
  </thead>
  <tbody></tbody>
</table>
```

That empty tbody is the first architectural decision. It keeps the page light and lets the CFC become the single source of truth for paging, sorting, filtering, and row shape.

DATATABLES INITIALIZATION IN INDEX.CFM

The client-side configuration turns on server-side processing and points DataTables at the ColdFusion component method. This is the point where the browser and ColdFusion agree on the contract.

```
let table = $('#employees').DataTable({
    autoWidth: true,
    processing: true,
    serverSide: true,
    rowId: 'employee_id',
    ajax: {
        url: 'com/Employees.cfc?method=getEmployeesJSON',
        type: 'POST'
    },
    paging: true,
    pageLength: 10,
    lengthMenu: [10, 25, 50, { label: 'All', value: -1 }]
});
```

processing: true gives users feedback while requests are in flight. **serverSide: true** tells DataTables not to perform filtering and sorting entirely in the browser. **rowId: employee_id** gives each row a stable DOM identity. The AJAX block points directly to the CFC endpoint using POST. **Paging** activates the pagination, while **pageLength** tells Datatable how many rows to display.

Notice the **length menu includes “All” with a value of -1**. DataTables uses -1 to mean all records. That is convenient in demos, but dangerous in production unless the server refuses to return an unlimited result set. Employees.cfc handles that with a maximum page-size cap, which we will cover shortly.

COLUMN MAPPING AND RENDERING

The columns array in index.cfm is more than display metadata. It is a client-side expression of the shape that the server returns. The CFC returns `employee_id`, `name`, `position`, `salary`, `office`, `extn`, and `start_date`. The DataTables configuration maps those fields onto visible columns.

- **employee_id** is numeric, searchable false, and orderable true.
- **name, position, office, and extn** are searchable and orderable.
- **salary** is rendered in the browser as US currency using `Intl.NumberFormat`.
- **start_date** is rendered as a long-localized date but is not orderable in this demo configuration.

```
{
  title: 'Salary',
  data: 'salary',
  type: 'num-fmt',
  searchable: true,
  orderable: true,
  className: 'dt-center fs-6',
  render: function (data, type, row) {
    let formattedAmount = new Intl.NumberFormat('en-US', {
      style: 'currency',
      currency: 'USD'
    }).format(data);
    return formattedAmount;
  }
}
```

This is a good separation of concerns. The database returns a numeric salary. ColdFusion returns it as data. The browser handles display formatting. That keeps the API useful to more than one possible consumer.

COLDFUSION JVM ARGUMENT MATCHING

Recent ColdFusion updates introduced stricter remote method argument validation in ColdFusion 2025 Update 2, ColdFusion 2023 Update 14, and ColdFusion 2021 Update 20.

This impacts DataTables because DataTables dynamically generates many request variables for searching, sorting, pagination, and column handling. Remote CFC methods must now explicitly declare incoming arguments or ColdFusion throw errors similar to:

```
coldfusion.runtime.UDFMethod$IllegalArgumentException
```

One method of fixing this issue is to declare a large number of DataTables-generated arguments in the remote method definition.

A second approach is to override the behavior with the following JVM argument:

```
-Dcoldfusion.runtime.remotemethod.matchArguments=false
```

If using CommandBox, add the setting to server.json:

```
"jvm":{  
  "args":[  
    "-Dcoldfusion.runtime.remotemethod.matchArguments=false"  
  ]  
}
```

One important detail: the leading hyphen in the JVM argument is required.

In the Employees CFC I use a third approach, that I'll discuss shortly.

THE COLDFUSION ENDPOINT

Now we move to `Employees.cfc`. The component begins with explicit tuning knobs: datasource name, maximum page size, and cache durations for total counts, filtered counts, unsearched pages, and searched pages.

```
variables.dsn          = "cf_user";
variables.maxPageSize   = 100;    // prevent request of 1,000,000 rows
variables.cacheTotalCount = createTimeSpan(0, 0, 10, 0); // 10 minutes
variables.cacheFiltered  = createTimeSpan(0, 0, 1, 0);  // 1 minute
variables.cachePageNoSearch = createTimeSpan(0, 0, 0, 30); // 30 seconds
variables.cachePageSearch  = createTimeSpan(0, 0, 0, 15); // 15 seconds
```

This is worth calling out because it shows the endpoint is not just a query wrapper. It is an API boundary with operational rules. The datasource is centralized, the maximum request size is centralized, and the caching strategy is visible at the top of the component instead of buried inside multiple query calls.

`Employees.cfc` also defines `onMissingMethod` for missing remote method calls. That prevents internal details from leaking to the client and logs the attempted method name and arguments for diagnostics.

THE `getEmployeesJSON()` function

```
var remote array function getEmployeesJSON(){
// DataTables posts a JSON body so ColdFusion's strict remote-argument
// matching does not reject the call before this method can run.
var dt = {
    "draw"    : 1,
    "start"    : 0,
    "length"  : 10
};
var http = getHttpRequestData();
var rawBody = trim(http.content);
dt = deserializeJSON(rawBody);
```

Now let's turn to the main function that orchestrates the API response, `getEmployeesJSON()`. As mentioned earlier, recent ColdFusion updates introduced stricter remote method argument validation, causing requests containing undeclared arguments to throw exceptions. While the JVM workaround restores the previous behavior, it also reopens a potential security concern. This is particularly challenging for DataTables, which generates several request variables for the table itself as well as additional variables for every column used in searching, sorting, and pagination. A cleaner solution is to bypass automatic argument binding entirely by reading the raw request body with `getHttpRequestData()` and converting the JSON payload into a ColdFusion structure. This approach eliminates the need to declare dozens of dynamic arguments, and works seamlessly with DataTables.

PAGING AND PAGE-SIZE SAFETY

DataTables sends `start` and `length`. `start` is zero-based. `length` is the requested page size. The server must treat both as untrusted input.

```
var pageStart = max(0, int(val(dt.start)));
var pageLength = int(val(dt.length ? : 10));
// length = -1 means "all"; we still cap it for safety.
if (pageLength EQ -1) {
    pageLength = variables.maxPageSize;
} else {
    pageLength = max(0, min(pageLength, variables.maxPageSize));
}
```

This is one of the most important security and performance lessons in the demo. The browser can ask for anything. A user can manipulate the request and ask for a million rows. The endpoint must make that impossible. This implementation allows the DataTables "All" option, but translates it into a safe maximum of 100 rows.

SECURE SORTING

Sorting is where many server-side DataTables examples become unsafe. **You cannot parameterize an ORDER BY column name with cfqueryparam or named SQL parameters.**

That means column names and directions must be controlled by whitelist logic before they ever reach the SQL string.

```
var orderDir = "asc";
if (structKeyExists(dt, "order[0][dir]")) {
    var requestedDir = lCase(trim(dt["order[0][dir]"]));
    if (listFindNoCase("asc,desc", requestedDir)) {
        orderDir = requestedDir;
    }
}
var orderColumn = "employee_id";
if (structKeyExists(dt, "order[0][column]")) {
    switch (int(val(dt["order[0][column]"]))) {
        case 0: orderColumn = "employee_id"; break;
        case 1: orderColumn = "name"; break;
        case 2: orderColumn = "position"; break;
        case 3: orderColumn = "salary"; break;
        case 4: orderColumn = "office"; break;
        case 5: orderColumn = "extn"; break;
        case 6: orderColumn = "start_date"; break;
        default: orderColumn = "employee_id";
    }
}
```

The browser sends a column index. The CFC translates that index into a known-safe database column. Unknown values fall back to `employee_id`. Direction is restricted to `asc` or `desc`. This is exactly the kind of small defensive pattern that prevents a demo from becoming a production incident.

SEARCH AND PARAMETERIZED QUERYEXECUTE

Global search is built with a WHERE clause and named QueryExecute parameters. The endpoint searches text fields using uppercase comparisons, date values using DATE_FORMAT, and numeric fields only when the search term is numeric.

```
var whereClause = "WHERE 1 = 1";
var params      = {};
var hasSearch   = false;
if (structKeyExists(dt, "search[value]") &&
    len(trim(dt["search[value]"]))) {
    hasSearch = true;
    var searchTerms = trim(dt["search[value]"]);
    var searchUpper = uCase(searchTerms);
    params.sLike = { value = "%#searchUpper#", cfsqltype = "cf_sql_varchar" };
    params.dLike = { value = "%#searchTerms#", cfsqltype = "cf_sql_varchar" };
    var isNum = isNumeric(searchTerms);
    if (isNum) {
        params.nLike =
            { value = "%#trim(searchTerms)#%", cfsqltype = "cf_sql_varchar" };
    }
}
```

The important point is not that every field is searched. The important point is that every user-provided search value is parameterized. The only dynamic SQL fragments are controlled fragments: the known WHERE clause structure, the whitelisted order column, the whitelisted direction, and the sanitized and parameterized page bounds.

COUNTS, CACHING, AND THE DATA QUERY

DataTables expects both total and filtered counts. Employees.cfc separates the total-count query into getAllEmployeesCount, caches it longer, and only runs a filtered count when a search is active.

```

var recordsTotal = getAllEmployeesCount();
var recordsFiltered = recordsTotal;
if (hasSearch) {
    var countSql = "SELECT COUNT(*) AS count FROM employees #whereClause#";
    var countOptions = {
        datasource    : variables.dsn,
        cachedWithin  : variables.cacheFiltered
    };
    var filtered = QueryExecute(countSql, params, countOptions);
    recordsFiltered = int(filtered.count);
}
params.pageLength = { value = pageLength, cfsqltype = "cf_sql_integer" };
params.pageStart  = { value = pageStart,  cfsqltype = "cf_sql_integer" };

```

The data query returns only the requested page and uses `returnType="array"`, which gives DataTables an array of structs that maps naturally onto the columns configuration in `index.cfm`.

```

var sql = "
    SELECT employee_id,
           name,
           position,
           salary,
           office,
           extn,
           DATE_FORMAT(start_date, '%m/%d/%Y') AS start_date
    FROM employees
    #whereClause#
    ORDER BY #orderColumn# #orderDir#
    LIMIT :pageLength OFFSET :pageStart ";
var dataOptions = {
    datasource    : variables.dsn,

```

```
returnType    : "array",
cachedWithin  : (hasSearch ? variables.cachePageSearch :
                  variables.cachePageNoSearch)
};
```

PageLength and pageStart are now fully parameterized before being passed to the database. While sanitizing and integer-capping these values before injecting them into the LIMIT and OFFSET clauses reduces risk, it's not ideal from a security perspective. By parameterizing the numeric values, this aligns with database security best practices and eliminates the need for direct value injection. This approach provides stronger protection against SQL injection, improves code maintainability, and demonstrates the importance of treating all user-supplied input as data rather than executable SQL.

THE DATATABLES JSON CONTRACT AND CLOSE

The final response is the DataTables contract: **draw**, **recordsTotal**, **recordsFiltered**, and **data**. If those keys are wrong, DataTables will not behave correctly. If the counts are wrong, pagination becomes confusing. If data is not an array of structs, the column mapping breaks.

```
var results = {
    "draw"          : int(val(dt.draw)),
    "recordsTotal"   : int(recordsTotal),
    "recordsFiltered" : int(recordsFiltered),
    "data"           : employees
};
return serializeJSON(results);
```

This is returned to the calling page which populates the table data returned from the `getEmployeesJSON()` function.

CLOSING MESSAGE

To conclude: ColdFusion can absolutely power fast, secure, server-side DataTables APIs. **The key is to stop treating the CFC as a loose remote method and start treating it as a real API boundary.**

- Validate the request.
- Whitelist anything that becomes SQL structure.
- Parameterize user input values.
- Cap pagination.
- Cache intelligently.
- Return the exact contract the browser expects.

KEY TAKEAWAYS

- Server-side DataTables is an API-design problem, not just a JavaScript plugin configuration.
- The browser request is untrusted, even when DataTables generated it.
- ORDER BY requires whitelisting because identifiers cannot be safely parameterized like values.
- Paging limits are a security control and a performance control.
- ColdFusion QueryExecute with named parameters keeps search values out of SQL injection territory.
- Caching should be tuned differently for total counts, filtered counts, searched pages, and unsearched pages.
- `ReturnType="array"` produces a JSON-friendly shape that DataTables can consume directly.

Thank you. I am happy to take questions.

OPTIONAL Q&A PROMPTS

- How would this change for SQL Server instead of MySQL?
- Should LIMIT/OFFSET be parameterized, and what happens when a driver does not support it cleanly?
- Where would column-specific searching fit into this endpoint?
- How would you add role-based filtering so users only see allowed rows?
- What indexes would you add for the demo employee search fields?
- Would you return structs directly and let ColdFusion serialize JSON, or manually serialize for stricter control?

LIVE DEMO CUES

- Open index.cfm and point out the empty tbody: rows are loaded over AJAX, not rendered server-side on initial page load.
- Show serverSide: true, processing: true, rowId: employee_id, and the POST endpoint URL.
- Use the browser Network tab to show DataTables sending draw, start, length, search[value], and order[0][column]/order[0][dir].
- Open Employees.cfc and show maxPageSize before discussing abuse prevention.
- Demonstrate how column index sorting maps to a switch statement instead of trusting browser-supplied column names.
- Search for a text value, then a numeric value, and explain how the WHERE clause changes for numeric searches.
- Close by showing the JSON object with draw, recordsTotal, recordsFiltered, and data.